Bridging the Enforcement Verification Gap: Detecting Divergence Between Declared and Actual Security Posture in Kubernetes Containerized Environments

Yehia Sobeh
Department of Cybersecurity
yehiasobehacm@gmail.com

ABSTRACT

Container orchestration platforms such as Kubernetes rely on declarative security primitives to enforce security boundaries around containerized workloads. A fundamental assumption underlies this model: that the container runtime faithfully translates declared security intent into kernel-level enforcement. This paper identifies and addresses the *enforcement verification gap*—the absence of tooling that systematically verifies whether declared Kubernetes security contexts are actually enforced by the container runtime. We present k8s-psa, a DaemonSet-based auditor that continuously compares pod security specifications retrieved from the Kubernetes API against the actual runtime state obtained through the container runtime interface. Through controlled experiments on a Minikube cluster with the Docker runtime, we evaluated the tool across seven test scenarios spanning fully hardened, partially secured, privileged, and deliberately misconfigured workloads. Our results reveal two categories of findings. First, privileged containers that declare seccomp system call filtering have that filtering silently stripped by the Docker runtime—a genuine enforcement gap that is invisible to existing specification-only tools. Second, the Docker runtime metadata interface omits the seccomp enforcement status of non-privileged containers that are correctly protected—an observability gap that produces false positives in naive runtime inspection. These findings demonstrate that the enforcement verification gap is real, that it manifests differently across container configurations, and that runtime inspection must account for the semantic differences between runtime metadata and actual kernel enforcement.

CCS CONCEPTS

• **Security and privacy** → **Virtualization and security**; *Access control*.

KEYWORDS

Kubernetes, container security, seccomp, runtime verification, enforcement drift, DevSecOps, security context

ACM Reference Format:
Yehia Sobeh. 2026. Bridging the Enforcement Verification Gap: Detecting Divergence Between Declared and Actual Security Posture in Kubernetes Containerized Environments. In *Cloud Computing Security Workshop (CCSW '26).* ACM, New York, NY, USA, 5 pages.

1 INTRODUCTION

Modern software systems increasingly adopt microservice architectures deployed as containerized workloads on Kubernetes. This shift introduces security challenges that traditional perimeter-based approaches cannot address: a larger attack surface distributed across potentially hundreds of containers, complex dependency chains that pull in third-party images, and dynamic runtime behavior in which containers are continuously created, scaled, and terminated [2, 23].

Kubernetes addresses these challenges through declarative security primitives specified in pod security contexts. Admission controllers such as OPA Gatekeeper [17] and Kyverno [11] intercept pod creation requests and reject non-compliant specifications, while Pod Security Admission (PSA), which has been stable since Kubernetes v1.25 [8], enforces predefined security profiles at the namespace level.

Despite their differences, these tools share a critical limitation: every one of them reads security properties from the Kubernetes API—that is, from the *declared* pod specification. None of them verifies that the container runtime has correctly translated those declarations into kernel-level enforcement. This verification matters because the translation from a Kubernetes pod specification to actual process isolation passes through multiple layers: the kubelet translates the pod specification into a Container Runtime Interface (CRI) request, the container runtime generates an Open Container Initiative (OCI) runtime specification, and a low-level runtime configures Linux kernel primitives [10]. Each translation step may add defaults, override values, or fail to apply certain restrictions.

This paper terms the resulting blind spot the *enforcement verification gap* and presents k8s-psa,[1] a tool designed to detect divergences between declared security intent and actual runtime enforcement.

1.1 Research Questions

- **RQ1:** To what extent does the actual security configuration of running containers diverge from the security properties declared in their Kubernetes pod specifications?
- **RQ2:** Can a lightweight, DaemonSet-based tool effectively detect such divergences with acceptable accuracy and performance overhead?
- **RQ3:** What categories of divergence occur in practice, and how should each category be classified from a security perspective?

CCSW '26, November 2026, The Hague, Netherlands
2026.

[1] Kubernetes Pod Security Auditor; not to be confused with the built-in Pod Security Admission (PSA) controller, which operates only on declared specifications.

1.2 Contributions

This work makes four contributions: (1) a novel verification tool that closes the enforcement verification gap by systematically comparing declared Kubernetes security contexts against observed container runtime state; (2) experimental evidence of genuine enforcement drift in privileged containers, where Docker silently strips declared seccomp profiles; (3) the discovery of an observability gap in the Docker runtime metadata interface, which causes correctly protected containers to appear unprotected; and (4) an open-source evaluation framework comprising 44 unit tests, 7 deployment-level test cases, and a real-time dashboard.

2 BACKGROUND

2.1 Kubernetes Security Architecture

Kubernetes enforces workload security through a layered model. The Kubernetes API accepts pod specifications containing `securityContext` fields that define the intended security posture [10]. The kubelet translates these specifications into CRI requests, from which the container runtime generates an OCI runtime specification [16]—a JSON document describing the exact Linux kernel configuration. A low-level runtime then configures namespaces, cgroups, capabilities, and seccomp filters.

2.2 Pod Security Admission and Policy Engines

Pod Security Admission, which has been stable since Kubernetes v1.25 [8], enforces Pod Security Standards at admission time but does not monitor running workloads or verify runtime enforcement. Policy engines extend this admission-time model with periodic re-evaluation: OPA Gatekeeper [17] audits cluster resources every 60 seconds, and Kyverno [11] performs background scans every hour. In both cases, however, the audit reads the declared pod specification from the Kubernetes API, and neither tool queries the container runtime.

2.3 Runtime Behavioral Monitoring

At the opposite end of the stack, behavioral monitors such as Falco [21] instrument the kernel via eBPF to detect anomalous system calls. These tools observe what containers *do* at the kernel level, but they do not verify whether the active security configuration matches what was declared.

3 RELATED WORK

3.1 Static Misconfiguration Detection

Research on Kubernetes security configuration has concentrated on the specification level. Rahman et al. [18] conducted a large-scale empirical study of 2,039 open-source Kubernetes manifests and explicitly identified runtime verification as future work. Building on the same specification-level view, Malul et al. [12] proposed GenKubeSec, a fine-tuned language-model approach to YAML misconfiguration detection, and Sgan Cohen et al. [19] presented KubeGuard, which analyzes configuration files and runtime logs with a focus on RBAC and NetworkPolicy hardening; both works, which have not yet undergone peer review, likewise operate on declared configuration rather than enforced state.

3.2 Configuration Drift Detection

A second line of work targets configuration drift. Kubediff [22] reports differences between the running Kubernetes state and version-controlled manifests. The Kubernetes blog [7] described admission controllers for detecting container drift at runtime, and Komodor [6] documented operational approaches to drift detection. All of these approaches, however, treat the Kubernetes API as the source of truth for the running state; none extends below the API to the container runtime layer.

3.3 The Identified Gap

Several results indicate that the layer below the API deserves scrutiny. Kermabon-Bobinnec et al. [5] demonstrated that temporal inconsistency between the orchestration and runtime layers can be exploited to bypass security policies, and Shamim et al. [20] concluded that security researchers should investigate dynamic configuration violation detection. Survey work by Wong et al. [23] and Devi Priya et al. [2] likewise identified runtime configuration drift detection as under-explored. To the best of our knowledge, however, no existing open-source tool systematically compares the security declarations held by the Kubernetes API against the state reported by the container runtime. The present work addresses precisely this gap.

4 METHODOLOGY

4.1 System Architecture

k8s-psa operates as a Kubernetes DaemonSet [9], deploying one scanner pod per cluster node. This deployment model guarantees complete node coverage, co-locates each scanner with the local container runtime socket, and scales automatically as nodes join or leave the cluster. Each instance performs periodic two-layer security scans; Fig. 1 shows the scanning pipeline.

Layer 1 (Spec Check) reads pod specifications from the Kubernetes API and checks them against eight configurable security policies (privileged mode, root user, host PID/network/IPC, seccomp profile, dangerous capabilities, and read-only root filesystem). This layer is functionally equivalent to existing policy engines and serves two purposes: it provides a baseline against which Layer 2 findings can be contextualized, and it flags pods that lack security declarations entirely—a case Layer 2 cannot detect, because the absence of a declaration leaves no intent to verify.

Layer 2 (Drift Detection)—the novel contribution—compares declared specifications against actual runtime state by: (1) extracting a `SpecSecurityState` from the Kubernetes API; (2) querying the container runtime (`docker inspect` or `crictl inspect`) to build a `RuntimeSecurityState`; (3) normalizing both states to account for expected divergences; and (4) comparing the normalized states field by field across seven security dimensions.

The comparison engine classifies each divergence as **CRITICAL** (a declared restriction is absent at runtime), **WARNING** (an undeclared capability is present), or **INFO** (the runtime is more restrictive than declared).

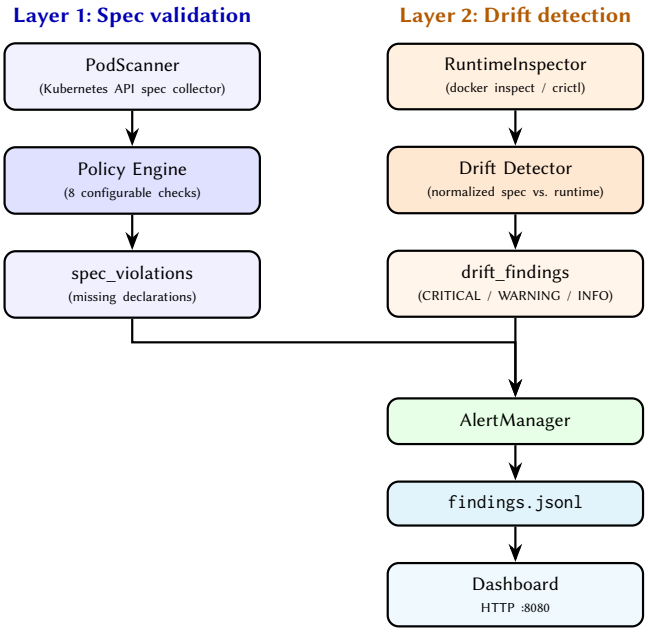


Figure 1: Two-layer scanning pipeline of k8s-psa. Layer 1 detects missing security declarations in pod specifications; Layer 2 (the novel contribution) detects divergence between declared and enforced state. Both streams feed the AlertManager, which produces NDJSON records consumed by the dashboard.

4.2 Capability Normalization

Naive comparison of declared and observed capability sets would flood the operator with false positives, because the container runtime adds 14 default Linux capabilities to every non-privileged container [3]. The normalizer therefore computes the *expected* capability set from the pod specification: it starts with these documented defaults, removes the capabilities listed in `drop`, and adds those listed in `add`. Only deviations from this expected set generate findings.

4.3 Experimental Environment

Experiments were conducted on a single-node Minikube cluster (v1.35.0) on Ubuntu 22.04, running Kubernetes v1.32.0 with Docker 27.4.1. The scanner interval was 60 seconds. Seven test pods were designed to cover the spectrum of security configurations—fully hardened, no security context, privileged with and without a declared seccomp profile, partially secured, privilege-escalation-restricted, and host-namespace-sharing—and three scan cycles were captured per pod. In addition to the seven purpose-built pods, the scanner observed the Kubernetes control plane pods running in the cluster, which serve as unplanned real-world subjects.

5 RESULTS

5.1 Detection Results Summary

Table 1 presents the Layer 2 drift findings for each test pod. Layer 1 behaved as expected across all seven pods (e.g., zero violations for the fully hardened Pod 1 and six violations for the host-namespace Pod 7) and is not discussed further; the remainder of this section focuses on the Layer 2 findings that constitute the contribution of this work.

Table 1: Layer 2 Drift Findings by Test Pod

Test Pod	CRIT	WARN	INFO	Match
1. Fully Hardened	1*	0	0	§5.3
2. No Security Ctx	0	0	0	Yes
3. Priv + Seccomp	1	27	0	Yes
4. Priv, No Seccomp	0	27	0	Yes
5. Partial Security	1*	0	0	§5.3
6. No Privesc	1*	0	0	§5.3
7. Host Namespaces	0	0	2	Yes

*Observability false positives analyzed in Section 5.3.

5.2 True Positive: Privileged Seccomp Stripping

The central finding concerns Test Pod 3, which declares both `privileged: true` and `seccompProfile.type: RuntimeDefault`. For such containers, the Docker runtime grants all 41 Linux capabilities and removes the seccomp filter entirely, even though seccomp filtering is explicitly declared. This behavior is documented in the Moby project issue tracker [14]. The tool reported the divergence as follows:

```
{
  "field": "seccompProfile",
  "severity": "critical",
  "category": "seccomp_mismatch",
  "description": "Spec declares seccomp type
    'RuntimeDefault' but the runtime has NO
    seccomp filter (unconfined)",
  "declared": "RuntimeDefault",
  "actual": "unconfined (null)"
}
```

Test Pod 4 serves as the experimental control: it has the same runtime state as Pod 3 but no seccomp declaration, and it produced zero CRITICAL findings. The 27 WARNING-level capability findings are identical between Pods 3 and 4, reflecting the 27 capabilities that privileged mode grants beyond the 14 defaults—including `CAP_SYS_ADMIN`, `CAP_NET_ADMIN`, and `CAP_SYS_MODULE`.

The security implication is direct. An operator who declares `RuntimeDefault` on a privileged container reasonably expects system call filtering to be active. Any specification-only tool reads the Kubernetes API, sees `RuntimeDefault`, and reports the container as filtered. Only by querying the container runtime does the silent removal become visible.

5.3 Observability Gap

Test Pods 1, 5, and 6—all non-privileged and all declaring `RuntimeDefault`—unexpectedly produced CRITICAL seccomp findings. Investigation revealed that these are false positives caused by a documented but widely misunderstood behavior of the Docker metadata interface. When cri-dockerd translates `RuntimeDefault`, it passes no explicit

`SecurityOpt` entry to Docker and instead relies on Docker's implicit default profile [3], which restricts several dozen sensitive system calls such as `mount`, `ptrace`, and `reboot` [13]. Because the implicit profile never appears in `docker inspect` output, a correctly protected container is indistinguishable—at the metadata level—from an unprotected one. The CIS Docker Benchmark (check 5.21) documents exactly this semantics: an empty `SecurityOpt` field indicates that the default profile is *active*, not that seccomp is disabled [1, 15].

The Kubernetes control plane pods observed during the evaluation (etcd, kube-apiserver, kube-controller-manager, kube-scheduler) exhibited the same phenomenon, each producing one false-positive CRITICAL seccomp finding. This consistency across purpose-built and real-world workloads confirms that the observability gap is systematic rather than an artifact of the test pod design.

5.4 Pod 3 vs. Pod 4: Controlled Comparison

Table 2 presents the controlled comparison between Pod 3 and Pod 4, which isolates the seccomp declaration as the single independent variable.

Table 2: Controlled Comparison: Pod 3 vs. Pod 4

Property	Pod 3	Pod 4
Spec: privileged	true	true
Spec: seccompProfile	RuntimeDefault	Not declared
Runtime: capabilities	All 41	All 41
Runtime: seccomp	Unconfined	Unconfined
L2 CRITICAL	**1**	**0**
L2 WARNING	27	27

Both pods have identical runtime states; the only difference is the seccomp declaration. The drift detector flags Pod 3 and remains silent on Pod 4, demonstrating its *specificity*: the tool distinguishes "restriction declared but not enforced" from "restriction never declared."

5.5 Performance Characteristics

Each scan cycle completed in approximately 3.1 seconds for 16 pods (0.7 s for Layer 1 and 2.4 s for Layer 2 across 18 containers), well within the 60-second scan interval. The scanner consumed 64 MiB of memory and fewer than 100m CPU, and the dashboard a further 32 MiB. The tool ships with 44 automated unit tests that execute in 0.2 seconds.

6 DISCUSSION

6.1 Answers to the Research Questions

RQ1 (extent of divergence). Divergence between declared and enforced security is not hypothetical. Privileged containers with seccomp declarations exhibit a genuine enforcement gap in which Docker silently removes the declared restriction, and privileged mode inflates the capability set by 27 capabilities beyond the declared expectation.

RQ2 (feasibility of lightweight detection). A DaemonSet-based tool can detect such divergences with modest resources (64 MiB, <100m CPU, 3.1 s per cycle). The controlled Pod 3/Pod 4 comparison shows that detection is specific rather than merely sensitive; the observability false positives show that accuracy depends on correctly interpreting runtime metadata semantics.

RQ3 (categories of divergence). Three categories emerged: (1) a *genuine enforcement gap* (CRITICAL)—a declared seccomp profile stripped by privileged mode; (2) *capability inflation* (WARNING)—privileged mode granting all 41 capabilities; and (3) an *observability gap* (false-positive CRITICAL)—Docker's implicit seccomp default being invisible to metadata inspection.

6.2 The Gap Is Real but Nuanced

The results therefore confirm the enforcement verification gap while revealing that it has two distinct forms. The enforcement gap creates a false sense of security: the declared control simply is not there. The observability gap complicates verification tool development and demonstrates that runtime metadata is a lossy abstraction over kernel state—a definitive verdict requires reading the kernel's own enforcement status from `/proc/<pid>/status`, where a `Seccomp` value of 2 confirms that BPF filter mode is active [4].

6.3 Comparison with Existing Tools

Table 3 positions k8s-psa in the existing tool landscape. To the best of our knowledge, no existing open-source tool performs the spec-versus-runtime comparison that k8s-psa implements: OPA Gatekeeper [17] and Kyverno [11] read only the Kubernetes API, Falco [21] monitors kernel behavior without spec comparison, and Kubediff [22] reports differences between the API state and git manifests. k8s-psa occupies the space between specification-level tools and kernel-level monitors.

Table 3: Positioning of k8s-psa Among Existing Tools

Tool	What It Checks	Layer
OPA Gatekeeper	Spec vs. policy constraints	API only
Kyverno	Spec vs. validate rules	API only
Falco	Syscall behavior (eBPF)	Kernel only
Kubediff	Spec vs. git manifests	API vs. git
k8s-psa	**Spec vs. runtime state**	**API vs. runtime**

6.4 Limitations

Five limitations bound the generality of these results. First, the experiments use Minikube with the Docker runtime, whereas production clusters predominantly use containerd, where the observability gap may not exist because `crictl inspect` exposes the full OCI runtime specification. Second, the seccomp verification relies on runtime metadata rather than kernel-level status. Third, the evaluation was conducted on a single-node cluster. Fourth, the tool shells out to CLI commands rather than using native API client libraries. Fifth, seven test pods cover the primary security dimensions but not every configuration (e.g., init, sidecar, or ephemeral containers).

6.5 Future Work

Three directions merit investigation: extending the detector with `/proc/<pid>/status` verification, which would eliminate the observability gap entirely; deploying on containerd-based clusters (kind, GKE, EKS) to compare enforcement behavior across runtimes; and investigating whether mutating admission webhooks introduce additional enforcement gaps by modifying security contexts after user submission.

7 CONCLUSION

This paper identified and addressed the enforcement verification gap in Kubernetes security. Through controlled experiments, we demonstrated that privileged containers declaring seccomp filtering had that filtering silently removed by the Docker runtime—a genuine enforcement failure that is invisible to all existing specification-only tools. We additionally discovered an observability gap in Docker's metadata interface that makes correctly protected containers appear unprotected. The tool, its evaluation framework, and its real-time dashboard are provided as open-source contributions.

REFERENCES

[1] Center for Internet Security. 2024. CIS Docker Benchmark v1.7.0. https://www.cisecurity.org/benchmark/docker. Accessed: 2026-07-10.

[2] Devi Priya V S, Sibi Chakkaravarthy Sethuraman, and Muhammad Khurram Khan. 2023. Container security: Precaution levels, mitigation strategies, and research perspectives. *Computers & Security* 135 (2023), 103490. https://doi.org/10.1016/j.cose.2023.103490

[3] Docker. 2024. Seccomp Security Profiles for Docker. https://docs.docker.com/engine/security/seccomp/. Accessed: 2026-07-10.

[4] Sascha Grunert and Mrunal Patel. 2023. KEP-2413: Enable Seccomp by Default. https://github.com/kubernetes/enhancements/tree/master/keps/sig-node/2413-seccomp-by-default. Kubernetes Enhancement Proposal. Accessed: 2026-07-10.

[5] Hugo Kermabon-Bobinnec, Mahmood Gholipourchoubeh, Sima Bagheri, Suryadipta Majumdar, Yosr Jarraya, Makan Pourzandi, and Lingyu Wang. 2022. ProSPEC: Proactive Security Policy Enforcement for Containers. In *Proceedings of the 12th ACM Conference on Data and Application Security and Privacy (CODASPY '22)*. 1–12. https://doi.org/10.1145/3508398.3511515

[6] Komodor. 2023. Drift Detection in Kubernetes. https://komodor.com/blog/drift-detection-in-kubernetes/. Accessed: 2026-07-10.

[7] Kubernetes. 2021. Using Admission Controllers to Detect Container Drift at Runtime. https://kubernetes.io/blog/2021/12/21/admission-controllers-for-container-drift/. Accessed: 2026-07-10.

[8] Kubernetes. 2022. Kubernetes v1.25: Pod Security Admission Controller in Stable. https://kubernetes.io/blog/2022/08/25/pod-security-admission-stable/. Accessed: 2026-07-10.

[9] Kubernetes. 2024. DaemonSet. https://kubernetes.io/docs/concepts/workloads/controllers/daemonset/. Accessed: 2026-07-10.

[10] Kubernetes. 2024. Restrict a Container's Syscalls with seccomp. https://kubernetes.io/docs/tutorials/security/seccomp/. Accessed: 2026-07-10.

[11] Kyverno. 2024. Background Scans. https://kyverno.io/docs/policy-reports/background/. Accessed: 2026-07-10.

[12] Ehud Malul, Yair Meidan, Dudu Mimran, Yuval Elovici, and Asaf Shabtai. 2024. GenKubeSec: LLM-Based Kubernetes Misconfiguration Detection, Localization, Reasoning, and Remediation. https://doi.org/10.48550/arXiv.2405.19954 arXiv:2405.19954

[13] Rory McCune. 2023. Container Security Fundamentals Part 6: Seccomp. https://securitylabs.datadoghq.com/articles/container-security-fundamentals-part-6/. Datadog Security Labs. Accessed: 2026-07-10.

[14] Moby. 2020. Allow for Custom Seccomp Profiles to Be Used in Privileged Containers. https://github.com/moby/moby/issues/41321. GitHub Issue #41321. Accessed: 2026-07-10.

[15] Zlatan Morić, Vedran Dakić, and Tomislav Čavala. 2025. Security Hardening and Compliance Assessment of Kubernetes Control Plane and Workloads. *Journal of Cybersecurity and Privacy* 5, 2 (2025), 30. https://doi.org/10.3390/jcp5020030

[16] Open Container Initiative. 2023. OCI Runtime Specification. https://github.com/opencontainers/runtime-spec/blob/master/config.md. Accessed: 2026-07-10.

[17] Open Policy Agent. 2024. Gatekeeper Audit. https://open-policy-agent.github.io/gatekeeper/website/docs/audit/. Accessed: 2026-07-10.

[18] Akond Rahman, Shazibul Islam Shamim, Dibyendu Brinto Bose, and Rahul Pandita. 2023. Security Misconfigurations in Open Source Kubernetes Manifests: An Empirical Study. *ACM Transactions on Software Engineering and Methodology* 32, 4 (2023), 99:1–99:36. https://doi.org/10.1145/3579639

[19] Omri Sgan Cohen, Ehud Malul, Yair Meidan, Dudu Mimran, Yuval Elovici, and Asaf Shabtai. 2025. KubeGuard: LLM-Assisted Kubernetes Hardening via Configuration Files and Runtime Logs Analysis. https://doi.org/10.48550/arXiv.2509.04191 arXiv:2509.04191

[20] Shazibul Islam Shamim, Hanyang Hu, and Akond Rahman. 2025. Dynamic Application Security Testing for Kubernetes Deployment: An Experience Report from Industry. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering (FSE '25), Industry Papers Track*. https://doi.org/10.1145/3696630.3728573

[21] Sysdig. 2024. Container Drift Detection with Falco. https://www.sysdig.com/blog/container-drift-detection-with-falco. Accessed: 2026-07-10.

[22] Weaveworks. 2020. Kubediff. https://github.com/weaveworks/kubediff. Accessed: 2026-07-10.

[23] Aaron Yi Wong, Eyasu Getahun Chekole, Martín Ochoa, and Jianying Zhou. 2023. On the security of containers: Threat modeling, attack analysis, and mitigation strategies. *Computers & Security* 128 (2023), 103140. https://doi.org/10.1016/j.cose.2023.103140